

Лабораторная работа № 9.

Загрузка файлов и защита пользовательского контента

Тема: Multer, multipart/form-data, безопасная проверка и хранение изображений Product

Цель: Реализовать безопасную загрузку одного JPEG/PNG изображения для товара Internet Shop с ограничением размера, allowlist-проверками, server-generated filename, signature validation, хранением metadata в PostgreSQL и контролируемой выдачей файла.

Дисциплина: Node.js и Express.js

Исходные условия

Продолжайте проект express-shop. В проекте должны работать Product API, Prisma/PostgreSQL, JWT, requireAuth, requireRole("admin"), validation middleware и централизованный error handler.

Назначение лабораторной

В работе разрешаются только JPEG и PNG изображения Product. Не расширяйте allowlist на документы, архивы, scripts или другие типы без отдельного анализа рисков.

1. Установите актуальный Multer

```
PowerShell  
  
npm install multer@^2.4.0
```

Не используйте старые версии

Для file upload особенно важно использовать актуальную поддерживаемую версию middleware с исправлениями безопасности.

2. Создайте storage directory

```
Структура проекта  
  
express-shop/  
  storage/  
    product-images/  
  middleware/  
  services/  
  controllers/  
  routes/  
  prisma/
```

Каталог storage/product-images не должен находиться внутри frontend/public directory.

3. Добавьте metadata изображения в Product

prisma/schema.prisma

```
model Product {
  id          Int      @id @default(autoincrement())
  name        String
  price       Int
  categoryId  Int      @map("category_id")
  inStock     Boolean @default(true) @map("in_stock")
  imageName   String? @map("image_name")
  imageMime   String? @map("image_mime")
  imageSize   Int?    @map("image_size")

  category Category @relation(fields: [categoryId], references: [id])

  @@map("products")
}
```

PowerShell

```
npx prisma@7 migrate dev --name add_product_image
npx prisma@7 generate
```

4. Создайте функцию безопасного имени

utils/fileNames.js

```
import { randomUUID } from "node:crypto";

export function createImageFilename(mimetype) {
  const extension = mimetype === "image/png"
    ? ".png"
    : ".jpg";

  return `${randomUUID()}${extension}`;
}
```

Итоговое имя не зависит от `file.originalname` и генерируется сервером.

5. Создайте Multer storage

middleware/uploadMiddleware.js

```
import multer from "multer";
import { mkdirSync } from "node:fs";
import { resolve } from "node:path";
import { createImageFilename } from "../utils/fileNames.js";

export const productImageDir = resolve(
  "storage/product-images"
);

mkdirSync(productImageDir, {
  recursive: true
});
```

```
const storage = multer.diskStorage({
  destination(req, file, callback) {
    callback(null, productImageDir);
  },

  filename(req, file, callback) {
    callback(null, createImageFilename(file.mimetype));
  }
});
```

6. Добавьте MIME allowlist

```
middleware/uploadMiddleware.js

const allowedMimeTypes = new Set([
  "image/jpeg",
  "image/png"
]);

function fileFilter(req, file, callback) {
  if (!allowedMimeTypes.has(file.mimetype)) {
    return callback(
      new Error("Unsupported file type")
    );
  }

  callback(null, true);
}
```

MIME ещё не доказательство

`file.mimetype` приходит из HTTP request и может быть подделан. Ниже будет дополнительная проверка фактических байтов файла.

7. Добавьте limits

```
middleware/uploadMiddleware.js

export const uploadProductImage = multer({
  storage,
  fileFilter,
  limits: {
    fileSize: 5 * 1024 * 1024,
    files: 1,
    fields: 5
  }
});
```

Разрешается один файл размером не более 5 MB.

8. Создайте signature validation

```
services/imageValidationService.js
```

```
import { open } from "node:fs/promises";

export async function detectImageType(filePath) {
  const file = await open(filePath, "r");
  const buffer = Buffer.alloc(8);

  try {
    await file.read(buffer, 0, 8, 0);
  } finally {
    await file.close();
  }

  const pngSignature = Buffer.from([
    0x89, 0x50, 0x4e, 0x47,
    0x0d, 0x0a, 0x1a, 0x0a
  ]);

  const isPng = buffer.equals(pngSignature);

  const isJpeg =
    buffer[0] === 0xff &&
    buffer[1] === 0xd8 &&
    buffer[2] === 0xff;

  if (isPng) return "image/png";
  if (isJpeg) return "image/jpeg";

  return null;
}
```

9. Создайте upload service

```
services/productImageService.js

import { unlink } from "node:fs/promises";
import { prisma } from "../config/prisma.js";
import { detectImageType } from "../imageValidationService.js";

export async function saveProductImage(
  productId,
  file
) {
  const detectedType = await detectImageType(file.path);

  if (!detectedType) {
    await unlink(file.path);

    const error = new Error("Invalid image content");
    error.status = 400;
    throw error;
  }

  const product = await prisma.product.findUnique({
    where: { id: productId }
  });

  if (!product) {
    await unlink(file.path);

    const error = new Error("Product not found");
  }
}
```

```

    error.status = 404;
    throw error;
  }

  return prisma.product.update({
    where: { id: productId },
    data: {
      imageName: file.filename,
      imageMime: detectedType,
      imageSize: file.size
    }
  });
}

```

Если signature не распознана или Product отсутствует, записанный файл удаляется.

10. Создайте Controller загрузки

```

controllers/productImageController.js

import { saveProductImage } from "../services/productImageService.js";

export async function uploadProductImageController(
  req,
  res,
  next
) {
  try {
    if (!req.file) {
      return res.status(400).json({
        error: "Image file is required"
      });
    }

    const product = await saveProductImage(
      req.validated.id,
      req.file
    );

    res.status(201).json({
      id: product.id,
      imageName: product.imageName,
      imageMime: product.imageMime,
      imageSize: product.imageSize
    });
  } catch (error) {
    next(error);
  }
}

```

11. Подключите POST upload route

```

routes/productRoutes.js

router.post(
  "/:id/image",
  requireAuth,
  requireRole("admin"),
  productIdValidation,

```

```
validateRequest,  
uploadProductImage.single("image"),  
uploadProductImageController  
);
```

Поле файла в `multipart/form-data` должно называться `image`.

12. Проверьте upload через frontend

Фрагмент frontend

```
<form id="image-form">  
  <input  
    id="image-input"  
    type="file"  
    name="image"  
    accept="image/jpeg,image/png"  
    required  
  >  
  <button type="submit">Загрузить</button>  
</form>
```

Frontend JavaScript

```
const form = document.querySelector("#image-form");  
const input = document.querySelector("#image-input");  
  
form.addEventListener("submit", async (event) => {  
  event.preventDefault();  
  
  const formData = new FormData();  
  formData.append("image", input.files[0]);  
  
  const response = await fetch(  
    "http://localhost:3000/api/products/1/image",  
    {  
      method: "POST",  
      headers: {  
        Authorization: `Bearer ${accessToken}`  
      },  
      body: formData  
    }  
  );  
  
  console.log(await response.json());  
});
```

Не задавайте Content-Type вручную

При отправке `FormData` browser сам формирует `Content-Type: multipart/form-data` вместе с `boundary`.

13. Проверьте допустимый JPEG

1. Войдите как admin.

2. Выберите небольшой JPEG.
3. Отправьте форму.
4. Проверьте status 201.
5. Проверьте `storage/product-images`.
6. Убедитесь, что `filename` является UUID, а не `original filename`.
7. Проверьте `image_name`, `image_mime`, `image_size` в `Product`.

14. Проверьте допустимый PNG

Повторите upload для PNG. В `Product metadata` должен сохраниться `image/png`.

15. Проверьте неподдерживаемый тип

Попробуйте загрузить `.txt`, `.js` или другой файл, не входящий в `allowlist`. Upload должен завершиться ошибкой до `Controller`.

16. Проверьте fake image

8. Создайте обычный текстовый файл.
9. Переименуйте его в `fake.jpg`.
10. Отправьте файл как `image upload`.
11. Если быстрый MIME filter пропустит его, `signature validation` должна вернуть 400.
12. Проверьте, что `fake file` удалён из `storage`.

17. Проверьте file size limit

Попробуйте загрузить изображение больше 5 MB. `Multer` должен отклонить файл по `LIMIT_FILE_SIZE`.

18. Создайте endpoint выдачи изображения

```
controllers/productImageController.js

import { resolve } from "node:path";
import { prisma } from "../config/prisma.js";
import { productImageDir } from "../middleware/uploadMiddleware.js";

export async function getProductImage(req, res, next) {
  try {
    const product = await prisma.product.findUnique({
      where: {
        id: req.validated.id
      },
      select: {
        imageName: true,
        imageMime: true
      }
    });

    if (!product?.imageName || !product.imageMime) {
      return res.status(404).json({
        error: "Image not found"
      });
    }

    const filePath = resolve(
      productImageDir,
      product.imageName
    );

    res.type(product.imageMime);
```

```
    res.sendFile(filePath);
  } catch (error) {
    next(error);
  }
}
```

routes/productRoutes.js

```
router.get(
  "/:id/image",
  productIdValidation,
  validateRequest,
  getProductImage
);
```

19. Проверьте controlled delivery

13. Откройте GET /api/products/1/image.
14. Убедитесь, что browser получает изображение.
15. Проверьте корректный Content-Type.
16. Убедитесь, что URL не содержит filesystem path.
17. Убедитесь, что original filename не используется в URL.

20. Добавьте обработку Multer errors

middleware/errorHandler.js

```
import multer from "multer";

export function errorHandler(error, req, res, next) {
  if (error instanceof multer.MulterError) {
    if (error.code === "LIMIT_FILE_SIZE") {
      return res.status(413).json({
        error: "Image is too large"
      });
    }

    return res.status(400).json({
      error: "Upload failed"
    });
  }

  if (error.message === "Unsupported file type") {
    return res.status(400).json({
      error: "Unsupported file type"
    });
  }

  console.error(error);

  res.status(error.status ?? 500).json({
    error: error.status && error.status < 500
      ? error.message
      : "Internal server error"
  });
}
```


21. Самостоятельное задание

18. Добавьте endpoint DELETE `/api/products/:id/image` только для admin.
19. Удалите physical file через `unlink()`.
20. Очистите `imageName`, `imageMime`, `imageSize` в Product.
21. Не используйте original filename как storage path.
22. Проверьте upload без JWT.
23. Проверьте upload JWT роли user.
24. Проверьте upload JPEG и PNG.
25. Проверьте txt/js upload.
26. Проверьте fake `.jpg`.
27. Проверьте файл больше 5 MB.
28. Проверьте GET image после успешного upload.
29. Проверьте 404 для Product без изображения.

Требования к результату

- Используется актуальный Multer 2.4.x.
- Upload endpoint принимает multipart/form-data.
- Разрешён только один file field `image`.
- Размер ограничен 5 MB.
- Разрешены только JPEG/PNG.
- Проверяется фактическая file signature.
- Stored filename генерируется через UUID.
- Storage находится вне frontend/public directory.
- Только admin может загружать или удалять Product image.
- Metadata изображения сохраняется в Product.
- Файл выдаётся через controlled GET endpoint.
- Multer errors обрабатываются безопасно.

Критерии оценивания

№	Критерий
1	Multer установлен и подключён только к upload route
2	Upload принимает field image
3	Используется fileSize limit
4	Используется files limit
5	Есть JPEG/PNG allowlist
6	Студент понимает, что MIME можно подделать
7	Реализована signature validation
8	Invalid file удаляется
9	Stored filename генерирует server
10	Upload storage находится вне public webroot
11	Product metadata сохраняется в PostgreSQL

№	Критерий
12	POST upload защищён admin authorization
13	GET image использует metadata, а не user path
14	Oversized upload возвращает безопасную ошибку
15	Студент объясняет defence in depth для file upload